The code for the balloting problem will be developed in four incremental steps so that you can to learn the smart contract development process. Also, this process will allow you to learn Solidity language features by example. The four incremental steps in the development are as follows:

1. *BallotV1* --Define the data structures for the smart contract, and test them.
2. *BallotV2* --Add the constructor and the function to change the state of voting.
3. *BallotV3* --Add the other functions of the smart contract and a modifier to illustrate the use of Solidity features for enabling trust.
4. *BallotV4* --Add the trust elements `require()`, `revert()`, and `assert()`, and the function access modifier.

Let's now list the users of the system, data assets, and transactions based on the problem statement and the use case analysis.

3.2.4 Users, assets, and transactions

Now you're ready to apply design principle 3. Recall from the problem statement that the goal is for users to choose one of the many available proposals by voting on them. Based on the use case analysis in figure 3.3, the following are the users of the ballot system: the chairperson, the voters (including the chairperson), and anybody who's interested in the outcome of the balloting process.

The data assets, in this case, are the proposals on which the voters are voting. You also need to keep track of the voters,